

Global Variables in C

- In a programming language, each variable has a particular scope attached to them. The scope is either local or global. This article will go through global variables, their advantages, and their properties.
- The Declaration of a global variable is very similar to that of a local variable. The only difference is that the global variable is declared outside any function.

```
// C program to show declaration of global variable
#include <stdio.h>

int x = 5; // global variable

int main() {

int y = 10; // local variable
    return 0;
}
```

Advantages of Global Variable

Global variables can be accessed by all the functions present in the program.

Only a one-time declaration is required.

Global variables are very useful if all the functions are accessing the same data.

```
#include <stdio.h>

int var1 = 10;

void func()
{
    printf("Printing global variable from func() -> %d \n", var1);
}
```

```
int main()
{
    // printing both variables
    printf("Printing global variable from main() -> %d \n", var1);

    func();

    return 0;
}
```

Printing global variable from main() -> 10
Printing global variable from func() -> 10

Updating the value of a global variable.


```
#include <stdio.h>
float pi = 3.14;
void square()
{
    pi = pi * pi;
}
int main()
{
    printf("The initial value of the variable pi is: %f \n", pi);
    pi++;
    printf("Value of pi updated using main() function is: %f \n", pi);
    square();
    printf("Value of pi updated using square() function is: %f", pi);
    return 0;
}
```

output : The initial value of the variable pi is: 3.140000
Value of pi updated using main() function is: 4.140000
Value of pi updated using square() function is: 17.139603

we created a global variable pi. We also created a function square() that does not take any argument as its input. However, it can change the value of pi because it is a global variable, and we could access pi in this function. In the main function, we printed the initial value of pi, i.e. 3.140000. Then, we incremented the value of pi and printed the updated value, i.e. 4.140000. Finally, we called the square() function. Because the new value of pi was 4.14, the square() function squared 4.14 and we got 17.139603 as the new value of pi.

Redefining a global variable.

```
#include <stdio.h>
```

```
int var = 7;
```

```
// trying to update the value  
of var
```

```
var = 3;
```

```
int main()
```

```
{
```

```
    printf("%d", var);
```

```
    return 0;
```

```
}
```

main.c:6:1: warning: data definition has no type or storage class

6 | var = 3;

| ^~~

- we can observe that C does not allow us to redefine the value of a global variable. When we tried to redefine the global variable, we got a redefinition of 'var' error. However, we could change the value of global variables in functions like we did in the previous example.

Assigning a global variable to another global variable outside a scope.

one global variable could assign value to another global variable. **This means we could not change or redefine the global variables in the global scope**, but we could use global variables in the global scope.

```
#include <stdio.h>
```

```
int var1 = 3;
```

```
// assigning var1 to another global variable
```

```
int var2 = var1;
```

```
int main()
```

```
{
```

```
    printf("%d", var2);
```

```
    return 0;
```

```
}
```

output ; 3

- **Note: We should always initialize global variables with constant values, or the compiler automatically assigns 0 to it.**
- **local variable have higher precedence than global variable**
- **The global variables, unlike the local variables, do not get destroyed with the end of a function. These are available for any function till the end of the execution of the program- thus named global.**

```
#include<stdio.h>
int x, y = 10; // declaration and initialization of global variables here
int main()
{
printf("The Global x is = %d\n", x);
printf("The Global y is = %d\n\n", y);
func_a();
func_b();
return 0;
}
void func_a()
{
printf("From the func_a() the Global x is = %d\n", x);
printf("From the func_a() the Global y is = %d\n\n", y);
}
void func_b()
{
int x = 5;
printf("Inside the func_b() x is = %d\n", x);
}
```

Output

The output obtained here will be:

The Global x is = 0

The Global y is = 10

From the func_a() the Global x is = 0

From the func_a() the Global y is = 10

Inside the func_b() x is = 5

You can notice that in line 4, x and y get declared as two of the global variables of the type int. Here, the variable x will get initialized automatically to 0. Then one can use variables like x and y inside any of the given functions. Also notice that inside the func_2() function, there exists a local variable that has the same name as the global variable. When we have a conflict between the local and global variables, then the precedence is for the local variable. That's why the value of the local variable x gets printed inside the func().

```
int a = 15;  
int a = 30;  
int main()  
{  
printf(“%d”, a);  
return 0;  
}
```

**The output obtained here would be:
error: redefinition of ‘a’**

Access Global Variable if there is a Local Variable with Same Name in C?

- local variables have higher precedence (priority) than global variables. So, if there is a local variable with the same name as a global variable, we will get access to the local variable when we call the variable name.
- We use the 'extern' keyword to access the global variable if there is a local variable with the same name. The keyword extern stands for external or we could also use :: scope resolution operator to use global variables.


```
#include <stdio.h>

int var = 3;

void func()
{
    // declaring a local variable
    int var = 70;

    printf("The value of local var is: %d \n", var);
    {
        extern int var;
        printf("The value of global var using extern keyword is: %d \n", var);
    }

    printf("The value of global var using Scope resolution operator is: %d \n", ::var);
}

int main()
{
    func();
    return 0;
}
```

The value of local var is: 70

The value of global var using extern keyword is: 3

The value of global var using Scope resolution operator is: 3

when we wrote `extern int var` inside the new scope, we were able to print the value of the global var, and when we used `::` with the variable name, we could also access the global variable.

Conclusion

- **A global variable is a variable that is defined outside of all the functions.**
- **Global variables can be accessed and modified by any function in C.**
- **Global variables can only be defined before the main() function.**
- **We can not redefine the value of a global variable in a global scope; however, we could access a global variable in the global scope.**
- **If a local variable and a global variable have the same name, we can access the global variable by using the extern keyword.**